Appointment Scheduler Technical Progress Report

Software Design 1 SDN150S | Project 12

Prepared for Management | 4 November 2026

https://github.com/ZuluBarbie/AppointmentScheduler

Group 12

Noluthando | 260874574

Zilungile Cwathi | 250513447

Phakamani Nicholas Dlamini | 261059742

Sekete Manthata | 260555770

Liyabona Ndevu | 260173436

Okuhle Ntweni | 260016039

Reneloe Wondeful Phakwago | 260633283

Lisakhanya Songqwaba | 261136437

1 Introduction

Day 3 supports appointment creation, editing, deletion, file storage and daily, weekly and monthly views. Reminders remain outstanding for Project 12 (Cape Peninsula University of Technology, 2026).

2 Sub questions

How can edits survive restarting? How can failed saves preserve records? How can one list produce three schedule views?

3 Discussion

Appointment validates immutable values. AppointmentBook manages records, storage and date filtering. SchedulerFrame extends JFrame and handles selection and input. Main opens it on Swing's event thread (Oracle, n.d.a). The paired NetBeans form opened successfully in Design view.

The incremental method retains Days 1 and 2 while adding Day 3. VS Code supports editing, NetBeans supports visual forms, and the JDK builds each version. GitHub records meaningful updates. Constructors initialise objects; private fields restrict access; loops, conditionals and switch statements implement the rules. Exceptions report input and file errors.

Changes are written to a temporary file before replacement, requesting an atomic move where supported (Oracle, n.d.b). Memory changes after replacement. An unreadable file blocks editing; a stale window cannot overwrite an already changed file. Use one window for each data file.

4 Design artefacts

4.1 Pseudocode

OPEN window and LOAD validated records
ON add or edit: VALIDATE fields; PREPARE changed list
WRITE temporary file; REPLACE data file; UPDATE memory and table
ON save error: SHOW message; KEEP previous records
ON delete: CONFIRM; SAVE list without selected record
ON view change: FILTER day, Monday to Sunday, or calendar month
SORT by date and start time; DISPLAY rows

4.2 UML class diagram

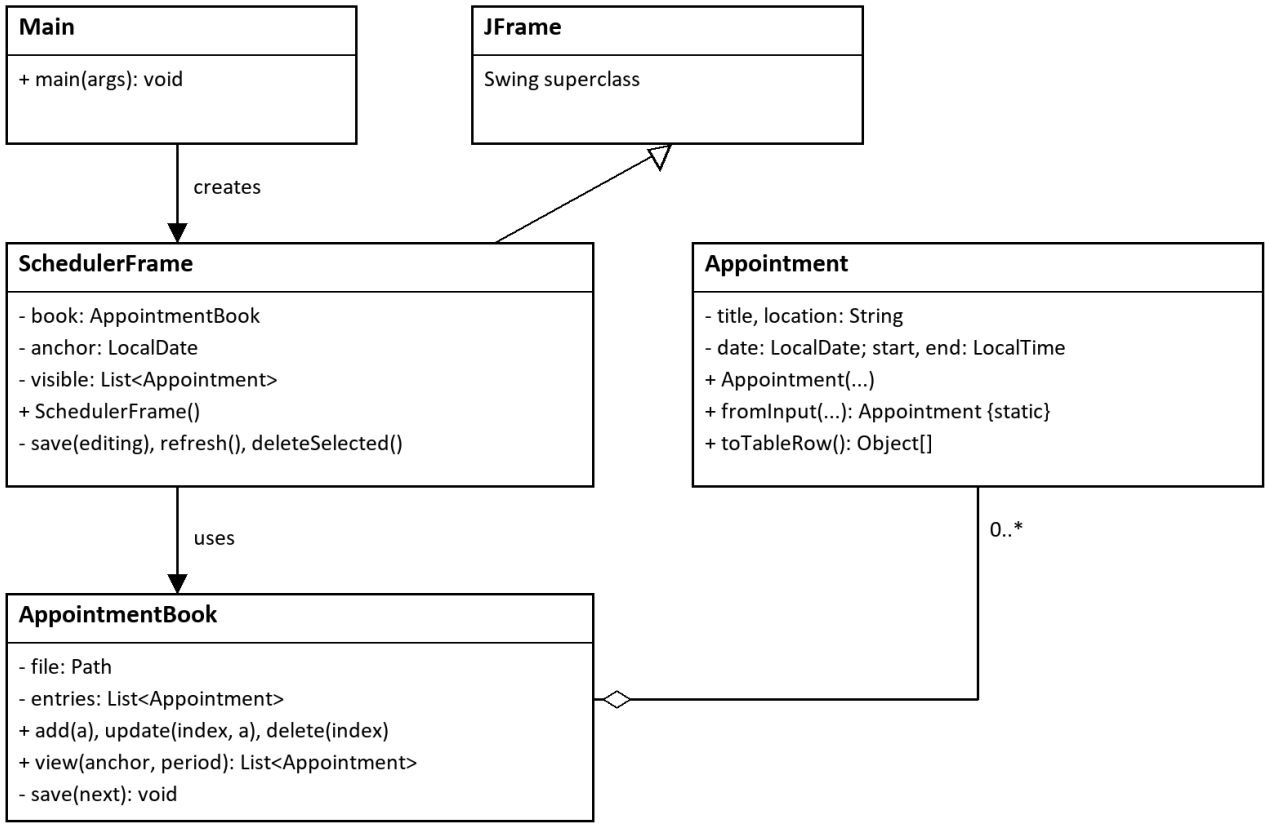


Figure 1. Day 3 classes; members abbreviated.

5 Results and analysis

All 38 automated checks passed, covering validation, persistence, edits, deletions, date boundaries, ordering, corruption and failed saves. Windows GUI checks verified three views, selection, editing and restart recovery using isolated data. NetBeans displayed the paired form. Deletion and failure recovery were verified by automated checks.

The file format preserves Unicode and separators. Storage is local and intended for one user. Reminders, recurrence, clash detection and calendar export remain planned.

6 Conclusion

Day 3 adds persistent editing and schedule views. Remaining functions, final report review and a 3-5 minute recording are due before submission on 8 November. Development targets 6 November.

References

Cape Peninsula University of Technology (2026) Software Design 1 SDN150S project brief. Course handout.

Oracle (n.d.a) SwingUtilities. Java SE 21. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/SwingUtilities.html (Accessed: 4 November 2026).

Oracle (n.d.b) Files. Java SE 21. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html (Accessed: 4 November 2026).

Appendix A Java program

Day 3 source listing. Entry point: demo.Main. The paired form and build scripts accompany the repository.

Main class

```
package demo;


import javax.swing.SwingUtilities;
import javax.swing.UIManager;

/** Opens the Day 3 appointment window on Swing's event thread. */
public class Main {
   public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> {
         try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
         } catch (Exception ex) {
            System.err.println("Using default appearance: " + ex.getMessage());
         }
         new SchedulerFrame().setVisible(true);
      });
   }
}
```

Appointment class

```
package demo;


import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;

/** A validated appointment with immutable values. */
public final class Appointment {
```

```
private static final DateTimeFormatter DATE_FORMAT = DateTimeFormatter
      .ofPattern("uuuu-MM-dd").withResolverStyle(ResolverStyle.STRICT);
private static final DateTimeFormatter TIME_FORMAT = DateTimeFormatter
      .ofPattern("HH:mm").withResolverStyle(ResolverStyle.STRICT);

private final String title;
private final LocalDate date;
private final LocalTime start;
private final LocalTime end;
private final String location;

public Appointment(String title, LocalDate date, LocalTime start, LocalTime end,
      String location) {
   if (title == null || title.isBlank()) {
      throw new IllegalArgumentException("Enter an appointment title.");
   }
   if (title.trim().length() > 60) {
      throw new IllegalArgumentException("Use no more than 60 characters for the title.");
   }
   if (date == null || start == null || end == null) {
      throw new IllegalArgumentException("Date, start time and end time are required.");
   }
   if (!end.isAfter(start)) {
      throw new IllegalArgumentException("End time must be after start time on the same day.");
   }
   this.title = title.trim();
   this.date = date;
   this.start = start;
   this.end = end;
   this.location = location == null ? "" : location.trim();
}

/** Parse form values strictly so invalid dates cannot silently change day. */
public static Appointment fromInput(String title, String date, String start,
```

```
            String end, String location) {
        LocalDate parsedDate;
        LocalTime parsedStart;
        LocalTime parsedEnd;
        try {
            parsedDate = LocalDate.parse(clean(date), DATE_FORMAT);
        } catch (DateTimeParseException ex) {
            throw new IllegalArgumentException("Enter a valid date as YYYY-MM-DD.");
        }
        try {
            parsedStart = LocalTime.parse(clean(start), TIME_FORMAT);
            parsedEnd = LocalTime.parse(clean(end), TIME_FORMAT);
        } catch (DateTimeParseException ex) {
            throw new IllegalArgumentException("Enter valid times as HH:mm, for example 09:00.");
        }
        return new Appointment(title, parsedDate, parsedStart, parsedEnd, location);
    }

    private static String clean(String value) { return value == null ? "" : value.trim(); }

    public String getTitle() { return title; }
    public LocalDate getDate() { return date; }
    public LocalTime getStart() { return start; }
    public LocalTime getEnd() { return end; }
    public String getLocation() { return location; }

    public Object[] toTableRow() {
        return new Object[]{title, date.toString(), start.format(TIME_FORMAT),
            end.format(TIME_FORMAT), location};
    }
}
```

AppointmentBook class

```
package demo;
```

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.time.*;
import java.util.*;

/** Holds validated records; a failed save leaves the previous list unchanged. */
public final class AppointmentBook {
   private final Path file;
   private byte[] savedBytes;
   private List<Appointment> entries = new ArrayList<>();

   public AppointmentBook(Path file) throws IOException {
      this.file = file.toAbsolutePath();
      if (Files.exists(this.file)) {
         savedBytes = Files.readAllBytes(this.file);
         List<String> lines = new String(savedBytes, StandardCharsets.UTF_8).lines().toList();
         if (lines.isEmpty() || !lines.get(0).equals("SCHEDULER-3")) {
            throw new IOException("Unrecognised appointment file: " + this.file);
         }
         try {
            for (int i = 1; i < lines.size(); i++) {
               String[] row = lines.get(i).split("\t", -1);
               if (row.length != 5) { throw new IllegalArgumentException("Invalid row"); }
               entries.add(new Appointment(decode(row[0]), LocalDate.parse(row[1]),
                     LocalTime.parse(row[2]), LocalTime.parse(row[3]), decode(row[4])));
            }
         } catch (RuntimeException ex) {
            throw new IOException("Appointment file contains invalid data: " + this.file, ex);
         }
      }
   }
```

```
public List<Appointment> getAll() { return List.copyOf(entries); }

public void add(Appointment appointment) throws IOException {
   Objects.requireNonNull(appointment);
   List<Appointment> next = new ArrayList<>(entries);
   next.add(appointment);
   save(next);
}

public void update(int index, Appointment appointment) throws IOException {
   Objects.requireNonNull(appointment);
   List<Appointment> next = new ArrayList<>(entries);
   next.set(index, appointment);
   save(next);
}

public void delete(int index) throws IOException {
   List<Appointment> next = new ArrayList<>(entries);
   next.remove(index);
   save(next);
}

/** Week views always run Monday through Sunday, including across years. */
public List<Appointment> view(LocalDate anchor, String period) {
   LocalDate from = switch (period) {
      case "Week" -> anchor.with(DayOfWeek.MONDAY);
      case "Month" -> anchor.withDayOfMonth(1);
      default -> anchor;
   };
   LocalDate to = switch (period) {
      case "Week" -> from.plusDays(6);
      case "Month" -> from.withDayOfMonth(from.lengthOfMonth());
      default -> from;
   };
```

```
    return entries.stream().filter(a -> !a.getDate().isBefore(from)
            && !a.getDate().isAfter(to))
            .sorted(Comparator.comparing(Appointment::getDate)
                    .thenComparing(Appointment::getStart)).toList();
}

private static String encode(String value) {
    return Base64.getEncoder().encodeToString(value.getBytes(StandardCharsets.UTF_8));
}

private static String decode(String value) {
    return new String(Base64.getDecoder().decode(value), StandardCharsets.UTF_8);
}

/** Replace the data file only after the complete new file has been written. */
private void save(List<Appointment> next) throws IOException {
    byte[] current = Files.exists(file) ? Files.readAllBytes(file) : null;
    if (!Arrays.equals(savedBytes, current)) {
        throw new IOException("Data changed in another window. Close and reopen this window.");
    }
    Path temporary = Files.createTempFile(file.getParent(), "appointments-", ".tmp");
    try {
        List<String> lines = new ArrayList<>();
        lines.add("SCHEDULER-3");
        for (Appointment a : next) {
            lines.add(String.join("\t", encode(a.getTitle()), a.getDate().toString(),
                    a.getStart().toString(), a.getEnd().toString(), encode(a.getLocation())));
        }
        Files.write(temporary, lines, StandardCharsets.UTF_8);
        try {
            Files.move(temporary, file, StandardCopyOption.ATOMIC_MOVE,
                    StandardCopyOption.REPLACE_EXISTING);
        } catch (AtomicMoveNotSupportedException ex) {
            Files.move(temporary, file, StandardCopyOption.REPLACE_EXISTING);
```

```
            }
            entries = next;
            savedBytes = Files.readAllBytes(file);
        } finally {
            Files.deleteIfExists(temporary);
        }
    }
}
```

SchedulerFrame class

```
package demo;

import java.awt.Color;
import java.io.IOException;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.List;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

/** Day 3 frame. The paired form describes the components for NetBeans Design. */
public class SchedulerFrame extends JFrame {
    private AppointmentBook book;
    private LocalDate anchor = LocalDate.now();
    private List<Appointment> visible = List.of();
    private final DefaultTableModel model = new DefaultTableModel(
            new String[]{"Title", "Date", "Start", "End", "Location"}, 0) {
        @Override public boolean isCellEditable(int row, int column) { return false; }
    };

    @SuppressWarnings("unchecked")
    public SchedulerFrame() {
        initComponents();
        ((JPanel) getContentPane()).setBorder(BorderFactory.createEmptyBorder(16,16,16,16));
```

```
        periodBox.setModel(new DefaultComboBoxModel<>(new String[]{"Day", "Week", "Month"}));
        anchorField.setColumns(10);
        table.setModel(model);
        table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        table.setFillsViewportHeight(true);
        table.setRowHeight(28);
        String[] labels = {"Title", "Appointment date", "Start time", "End time", "Location", "View date"};
        JTextField[] inputs = {titleField,dateField,startField,endField,locationField,anchorField};
        for (int i=0;i<inputs.length;i++) { inputs[i].getAccessibleContext().setAccessibleName(labels[i]); }
        table.getAccessibleContext().setAccessibleName("Appointments");
        addButton.addActionListener(e -> save(false));
        updateButton.addActionListener(e -> save(true));
        deleteButton.addActionListener(e -> deleteSelected());
        clearButton.addActionListener(e -> clear());
        table.getSelectionModel().addListSelectionListener(e -> { if (!e.getValueIsAdjusting()) select(); });
        showButton.addActionListener(e -> showDate());
        anchorField.addActionListener(e -> showDate());
        periodBox.addActionListener(e -> refresh());
        previousButton.addActionListener(e -> move(-1));
        nextButton.addActionListener(e -> move(1));
        todayButton.addActionListener(e -> { anchor=LocalDate.now(); refresh(); });
        try {
            book = new AppointmentBook(Path.of(System.getProperty("scheduler.dataFile","appointments.tsv")));
        } catch (IOException ex) {
            addButton.setEnabled(false);
            message("Cannot load data. File preserved: " + ex.getMessage(),true);
            JOptionPane.showMessageDialog(this,ex.getMessage(),"Could not load
appointments",JOptionPane.ERROR_MESSAGE);
        }
        clear();
        refresh();
        if (book != null) message("Loaded " + book.getAll().size() + " appointments. Changes save
automatically.",false);
        setMinimumSize(new java.awt.Dimension(1000,440));
```

```
    setSize(1120,560);
    setLocationRelativeTo(null);
}

private void message(String text, boolean error) {
    statusLabel.setForeground(error ? new Color(170,20,20) : new Color(0,90,35));
    statusLabel.setText(text);
}

private void showDate() {
    try { anchor=LocalDate.parse(anchorField.getText().trim()); refresh(); }
    catch (RuntimeException ex) { message("Enter a valid view date as YYYY-MM-DD.",true); }
}

private void move(int direction) {
    String period=(String)periodBox.getSelectedItem();
    anchor=switch(period) {
        case "Week" -> anchor.plusWeeks(direction);
        case "Month" -> anchor.plusMonths(direction);
        default -> anchor.plusDays(direction);
    };
    refresh();
}

private void refresh() {
    anchorField.setText(anchor.toString());
    visible=book==null ? List.of() : book.view(anchor,(String)periodBox.getSelectedItem());
    model.setRowCount(0);
    for (Appointment a:visible) model.addRow(a.toTableRow());
    updateButton.setEnabled(false);
    deleteButton.setEnabled(false);
}

private void select() {
```

```
    int row=table.getSelectedRow();
    boolean selected=row>=0 && row<visible.size();
    updateButton.setEnabled(selected && book!=null);
    deleteButton.setEnabled(selected && book!=null);
    if (!selected) return;
    Object[] values=visible.get(row).toTableRow();
    JTextField[] fields={titleField,dateField,startField,endField,locationField};
    for(int i=0;i<fields.length;i++) fields[i].setText(values[i].toString());
}

private void clear() {
    table.clearSelection();
    titleField.setText(""); locationField.setText("");
    dateField.setText(anchor.toString()); startField.setText("09:00"); endField.setText("10:00");
}

private void save(boolean editing) {
    if(book==null) return;
    int row=table.getSelectedRow();
    if(editing && row<0) { message("Select an appointment first.",true); return; }
    try {
        Appointment a=Appointment.fromInput(titleField.getText(),dateField.getText(),
                startField.getText(),endField.getText(),locationField.getText());
        if(editing) book.update(book.getAll().indexOf(visible.get(row)),a);
        else book.add(a);
        anchor=a.getDate(); refresh(); clear();
        message((editing ? "Updated" : "Added") + ": " + a.getTitle()+". Saved. Total: "+book.getAll().size(),false);
    } catch (IllegalArgumentException | IOException ex) { message("Not saved: "+ex.getMessage(),true); }
}

private void deleteSelected() {
    int row=table.getSelectedRow();
    if(book==null || row<0) return;
    Appointment a=visible.get(row);
```

```
    if(JOptionPane.showConfirmDialog(this,"Delete " + a.getTitle() + "?", "Delete appointment",
            JOptionPane.YES_NO_OPTION)!=JOptionPane.YES_OPTION) return;
    try {
        book.delete(book.getAll().indexOf(a)); refresh(); clear();
        message("Deleted: " + a.getTitle() + ". Saved.",false);
    } catch(IOException ex) { message("Not deleted: "+ex.getMessage(),true); }
}

@SuppressWarnings({"unchecked", "rawtypes"})
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
private void initComponents() {
    topPanel = new javax.swing.JPanel();
    topPanel.setLayout(new java.awt.BorderLayout(8, 8));
    navigationPanel = new javax.swing.JPanel();
    navigationPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
    viewLabel = new javax.swing.JLabel();
    viewLabel.setText("View");
    navigationPanel.add(viewLabel);
    periodBox = new javax.swing.JComboBox();
    navigationPanel.add(periodBox);
    anchorLabel = new javax.swing.JLabel();
    anchorLabel.setText("Date (YYYY-MM-DD)");
    navigationPanel.add(anchorLabel);
    anchorField = new javax.swing.JTextField();
    anchorField.setText("");
    navigationPanel.add(anchorField);
    showButton = new javax.swing.JButton();
    showButton.setText("Show");
    navigationPanel.add(showButton);
    previousButton = new javax.swing.JButton();
    previousButton.setText("Previous");
    navigationPanel.add(previousButton);
    todayButton = new javax.swing.JButton();
    todayButton.setText("Today");
```

```
navigationPanel.add(todayButton);
nextButton = new javax.swing.JButton();
nextButton.setText("Next");
navigationPanel.add(nextButton);
topPanel.add(navigationPanel, java.awt.BorderLayout.NORTH);
fieldsPanel = new javax.swing.JPanel();
fieldsPanel.setLayout(new java.awt.GridLayout(2, 5, 10, 6));
titleLabel = new javax.swing.JLabel();
titleLabel.setText("Title (required)");
fieldsPanel.add(titleLabel);
dateLabel = new javax.swing.JLabel();
dateLabel.setText("Date (YYYY-MM-DD)");
fieldsPanel.add(dateLabel);
startLabel = new javax.swing.JLabel();
startLabel.setText("Start (HH:mm)");
fieldsPanel.add(startLabel);
endLabel = new javax.swing.JLabel();
endLabel.setText("End (HH:mm)");
fieldsPanel.add(endLabel);
locationLabel = new javax.swing.JLabel();
locationLabel.setText("Location (optional)");
fieldsPanel.add(locationLabel);
titleField = new javax.swing.JTextField();
titleField.setText("");
fieldsPanel.add(titleField);
dateField = new javax.swing.JTextField();
dateField.setText("");
fieldsPanel.add(dateField);
startField = new javax.swing.JTextField();
startField.setText("");
fieldsPanel.add(startField);
endField = new javax.swing.JTextField();
endField.setText("");
fieldsPanel.add(endField);
```

```
    locationField = new javax.swing.JTextField();
    locationField.setText("");
    fieldsPanel.add(locationField);
    topPanel.add(fieldsPanel, java.awt.BorderLayout.CENTER);
    actionsPanel = new javax.swing.JPanel();
    actionsPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));
    addButton = new javax.swing.JButton();
    addButton.setText("Add appointment");
    actionsPanel.add(addButton);
    updateButton = new javax.swing.JButton();
    updateButton.setText("Save changes");
    actionsPanel.add(updateButton);
    deleteButton = new javax.swing.JButton();
    deleteButton.setText("Delete selected");
    actionsPanel.add(deleteButton);
    clearButton = new javax.swing.JButton();
    clearButton.setText("Clear selection");
    actionsPanel.add(clearButton);
    topPanel.add(actionsPanel, java.awt.BorderLayout.SOUTH);
    getContentPane().add(topPanel, java.awt.BorderLayout.NORTH);
    scrollPane = new javax.swing.JScrollPane();
    table = new javax.swing.JTable();
    scrollPane.setViewportView(table);
    getContentPane().add(scrollPane, java.awt.BorderLayout.CENTER);
    statusLabel = new javax.swing.JLabel();
    statusLabel.setText("Select an appointment to edit or delete.");
    getContentPane().add(statusLabel, java.awt.BorderLayout.SOUTH);
    setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
    setTitle("Appointment Scheduler - Day 3");
    pack();
}// </editor-fold>//GEN-END:initComponents

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JPanel topPanel;
```

```
    private javax.swing.JPanel navigationPanel;
    private javax.swing.JLabel viewLabel;
    private javax.swing.JComboBox periodBox;
    private javax.swing.JLabel anchorLabel;
    private javax.swing.JTextField anchorField;
    private javax.swing.JButton showButton;
    private javax.swing.JButton previousButton;
    private javax.swing.JButton todayButton;
    private javax.swing.JButton nextButton;
    private javax.swing.JPanel fieldsPanel;
    private javax.swing.JLabel titleLabel;
    private javax.swing.JLabel dateLabel;
    private javax.swing.JLabel startLabel;
    private javax.swing.JLabel endLabel;
    private javax.swing.JLabel locationLabel;
    private javax.swing.JTextField titleField;
    private javax.swing.JTextField dateField;
    private javax.swing.JTextField startField;
    private javax.swing.JTextField endField;
    private javax.swing.JTextField locationField;
    private javax.swing.JPanel actionsPanel;
    private javax.swing.JButton addButton;
    private javax.swing.JButton updateButton;
    private javax.swing.JButton deleteButton;
    private javax.swing.JButton clearButton;
    private javax.swing.JScrollPane scrollPane;
    private javax.swing.JTable table;
    private javax.swing.JLabel statusLabel;
    // End of variables declaration//GEN-END:variables
}
```